SolariNet Certificate Operations

Candidate operations manual · refreshed 23 October 2026 · F-01 CRL lifecycle

> **Candidate only — do not enable a rollout from this manual yet.** The generator, publisher, peer updater, acknowledgement path, and dashboard view exist in the local candidate. The HTTPS endpoint, host configuration, peer activation, rollout policy, and deployment are not installed or verified. This document describes the reviewed procedure for a later staged rollout; it is not evidence of a live service.

1. What a revocation means

The MariaDB revocation ledger is authoritative. A ledger entry alone does not change a TLS handshake. Effective revocation requires a fresh CA-signed CRL, authenticated HTTPS distribution, validation and activation on every relevant peer, and a subsequent authenticated report receipt. The current candidate receipt proves only that the reporting client or monitor agent included the local updater's activation marker. It does not attest every service on that host.

Keep the immutable root bundle separate from the generated active bundle. Never copy the CA private key to a publisher, proxy, dashboard, or peer. Do not expose a CRL endpoint over plaintext HTTP.

The dashboard counts only client and monitor reporting agents. It does not prove all host services loaded the CRL.

2. Components and access boundaries

Component	Runs as / reads	Writes or authority
CRL generator	`solari`; CA key, CA certificate, revocation ledger, issuance-inventory gate	Signs a bounded-lifetime CRL and advances per-issuer generation state
Static publisher	Dedicated unprivileged account; generated CRL and public CA certificate	Publishes only a verified issuer-fingerprint-named CRL
Peer updater	Root oneshot; trusted HTTPS endpoint, immutable roots, issuer certificate	Replaces one active bundle and restarts one allowlisted SolariNet service
Reporter/ server	Existing authenticated durable report; enrolled certificate identity	Stores latest exact-generation receipt; receipt DB failure does not reject the persisted report
Dashboard	Read-only API; generated generation, node state and receipt rows	Displays latest generation and client/monitor receipt status

3. Preconditions before any installation

1. Apply and verify the candidate migration set, including migration 035, using the approved migration process. Reconcile and attest the certificate issuance inventory before generating a CRL.
2. Confirm the issuer certificate and private key match. Keep the key owned by `solari` with mode `0600`. Do not grant the publisher, peer updater, web server, or dashboard access to it.
3. Establish a peer-trusted HTTPS hostname and certificate. Verify an unauthenticated peer GET serves exactly the generated CRL, and that the same path is unavailable over HTTP.
4. Review the exact service allowlist, active bundle paths, systemd sandbox, database grants, rollback behavior, and recovery access for each target host.
5. Use a staged, non-production peer first. Keep rollout policy fail-closed until the intended missing-CRL behavior is separately tested and approved.

4. Database accounts

Dashboard read account

The dashboard's read-only DB account also needs these CRL table reads in addition to its existing `node` privileges:

```
GRANT SELECT ON solarinet.certificateCrlState TO 'SOLARINET_DASHBOARD'@'DASHBOARD_HOST';
GRANT SELECT ON solarinet.certificateCrlAcknowledgement TO 'SOLARINET_DASHBOARD'@'DASHBOARD_HOST';
```

Server receipt account

Add only these rights to the existing SolariNet server account after migration 035 is present. Replace both account placeholders on the database host; do not paste secrets into commands or repository files.

```
GRANT SELECT ON solarinet.certificateCrlState TO 'SOLARINET_SERVER'@'SERVER_HOST';
GRANT SELECT, INSERT, UPDATE ON solarinet.certificateCrlAcknowledgement TO
'SOLARINET_SERVER'@'SERVER_HOST';
```

Acknowledgement persistence is best-effort. Missing migration or rights are logged and later reports retry the local receipt. The dashboard remains pending or stale until a valid matching row is recorded.

Generator account

```
GRANT SELECT ON solarinet.certificateInventoryState TO 'solari_crl'@'CRL_HOST';
GRANT SELECT ON solarinet.certificateRevocation TO 'solari_crl'@'CRL_HOST';
GRANT SELECT, INSERT, UPDATE ON solarinet.certificateCrlState TO 'solari_crl'@'CRL_HOST';
```

Use the narrowest DB host identity available and a generated password. Store client credentials in `/etc/solarinet/crl-generator.my.cnf`, readable only by `solari`:

```
[client]
host=DB_HOST
port=3306
user=solari_crl
password=REPLACE_ON_HOST_ONLY
```

```
sudo install -d -o root -g solari -m 0750 /etc/solarinet
sudoedit /etc/solarinet/crl-generator.my.cnf
sudo chown root:solari /etc/solarinet/crl-generator.my.cnf
sudo chmod 0640 /etc/solarinet/crl-generator.my.cnf
```

5. Install the CA-side generator and publisher

Install only reviewed release files. SolariNet C server daemon connections initialize their MariaDB session timezone to UTC on connect and reconnect; the generator uses UTC for ledger reads and generation writes. Xenon's active monitoring MariaDB was observed in `SYSTEM` mode with the host on UTC, and its `NOW()` matched `UTC_TIMESTAMP()`; no historical timestamp offset was observed there. Verify the actual configured database host before deployment. The generator service runs every six hours; a separate one-minute publisher timer exposes the generated CRL. Pre-publication failures preserve the previous CRL. There is a short non-transactional boundary after the atomic CRL replacement and before the generator advances the DB generation row; if that DB commit fails, the file can be one CRL number ahead of acknowledgement state.

```
sudo install -d -o root -g root -m 0755 /usr/local/libexec/solarinet
sudo install -o root -g root -m 0755 deploy/crl/generate_crl.py /usr/local/libexec/solarinet/
generate_crl.py
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-generate.service /etc/systemd/system/
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-generate.timer /etc/systemd/system/
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-publisher.sysusers.conf /etc/sysusers.d/
solari-crl-publisher.conf
sudo systemd-sysusers /etc/sysusers.d/solari-crl-publisher.conf
sudo install -o root -g root -m 0755 deploy/crl/publish_crl.py /usr/local/libexec/solarinet/
publish_crl.py
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-publish.service /etc/systemd/system/
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-publish.timer /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemd-analyze verify /etc/systemd/system/solari-crl-generate.service /etc/systemd/system/
solari-crl-generate.timer /etc/systemd/system/solari-crl-publish.service /etc/systemd/system/solari-
crl-publish.timer
```

Before enabling timers, check schema and inventory readiness, file ownership, account grants, and key/certificate match. Run one generation and inspect the issuer fingerprint, number, revoked count, CRL fingerprint, and expiry:

```
sudo systemctl start solari-crl-generate.service
sudo systemctl status solari-crl-generate.service --no-pager
sudo openssl crl -in /var/lib/solari-crl/current.pem -noout -verify \
  -CAfile /var/lib/solarinet/pki/ca.pem -crlnumber -lastupdate -nextupdate
sudo systemctl enable --now solari-crl-generate.timer solari-crl-publish.timer
```

Configure a static-only path inside a peer-trusted HTTPS virtual host using `deploy/crl/apache-crl-location.conf.example` or an equivalent rule. Do not expose directory listings, dotfiles, signing material, or database credentials.

6. Configure a peer updater

One updater instance handles one issuer and one service. Keep the immutable base bundle free of CRLs and include the issuer certificate. The timer first runs two minutes after boot or at the next hour with up to five minutes of jitter. Seed the active bundle from the configured base bundle before setting the service's `tls.caFile`; otherwise an early service restart can fail because the file does not yet exist. The per-service configuration contains public paths and a URL, not a secret:

```
CRL_URL=https://crl-host.example/solarinet/crl/ISSUER_SHA256.crl.pem
BASE_CA_BUNDLE=/etc/solarinet/trust-roots.pem
ISSUER_CA=/etc/solarinet/issuing-ca.pem
SOLARI_SERVICE=solari-client.service
```

Replace the issuer placeholder with the lowercase SHA-256 fingerprint printed by the generator. The endpoint hostname must match its HTTPS certificate. Seed the active bundle using the same base bundle path, then configure `tls.caFile` to the active path. Install the updater only after the endpoint has been independently verified from the peer.

The updater refuses every HTTP redirect. It uses the operating system CA store by default. For direct invocations, `--https-ca-file` selects an exclusive trust file; it does not add to the system roots.

```
sudo install -d -o root -g root -m 0755 /var/lib/solari-crl-active
sudo install -o root -g root -m 0644 /etc/solarinet/trust-roots.pem /var/lib/solari-crl-active/
client.pem
sudo install -d -o root -g root -m 0755 /usr/local/libexec/solarinet
sudo install -o root -g root -m 0755 deploy/crl/activate_crl.py /usr/local/libexec/solarinet/
activate_crl.py
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-activate@.service /etc/systemd/system/
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-activate@.timer /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemd-analyze verify /etc/systemd/system/solari-crl-activate@.service /etc/systemd/system/
solari-crl-activate@.timer
sudo systemctl enable --now solari-crl-activate@client.timer
```

The updater verifies HTTPS, issuer, signature, expiry, and monotonic number; rejects rollback or conflicting same-number CRLs; limits files to 4 MiB and validation to 120 seconds; serializes work per active bundle; and restarts only an allowlisted service. If restart/active verification fails, it restores the previous bundle and attempts recovery. A failed rollback requires manual intervention. The activation service has a 360-second outer timeout to leave 240 seconds for bounded recovery.

7. Verify reports and dashboard status

After a successful restart, the updater atomically writes `<active-bundle>.activated` as `issuer-sha256:number:crl-sha256`, mode `0644`. The client/monitor reporter includes this public marker in its authenticated durable report. The server records it only for the latest generated, unexpired generation. Stale, malformed, or unwriteable optional receipts do not reject the ordinary report.

The Certificates screen shows the latest generated issuer and CRL number, expiry, and peer receipt states. **Current** means an exact issuer/number/fingerprint receipt for an unexpired generation and a check-in within 15 minutes. **Expired generation** means the receipt matches the latest CRL, but that CRL has passed its next-update time or the expiry value is missing. **Stale** means a prior receipt does not match the latest generation. **Pending** means a recently seen peer has no receipt. **Offline** means no check-in within 15 minutes or node state is down. The denominator includes client and monitor nodes whose current leaf issuer matches that generation. Excluded agents are listed separately: an unknown issuer means no current leaf issuer is recorded; a different issuer means it is recorded but does not match. Server-role peers and other services on each host remain outside this status view.

> **Interpret carefully:** a current marker is evidence from one reporting agent, not cryptographic proof of runtime process state or host-wide convergence. A generated CRL is not necessarily already served, and a served CRL is not proof it was activated.

8. Recovery and rollback

Observed result	Operator action
Generator fails before output replacement	Read the systemd journal. Check ledger completeness, migration state, CA key/certificate match, DB rights, and system clock. Do not replace the last good CRL manually.
CRL file is one number ahead of DB state	A DB commit may have failed just after atomic file replacement. Restore DB availability, rerun the generator to allocate a higher number, and verify the output file and `certificateCrlState` agree before reading dashboard status.
Publisher fails or endpoint returns an old object	Verify generator output and publisher service/timer. Confirm the HTTPS static path serves the expected fingerprint-named file. Do not enable peer timers until this is corrected.
Updater rejects signature, issuer, number, or expiry	Keep the current trust bundle. Inspect endpoint URL, issuer fingerprint, current CRL metadata, and system clock; correct the source instead of bypassing validation.
Service restart fails	The updater restores the prior bundle and retries the prior service state. Inspect unit logs and verify the active bundle. If rollback reports failure, stop automation and restore the known-good bundle manually before restarting.
Dashboard shows pending/ stale	Check peer check-in, receipt file ownership/mode, reporter config, server migration 035 and grants, and whether the peer has fetched the latest generated generation.
Dashboard says acknowledgement data unavailable	Check API logs, migration 035, and read access. The API fails soft; this state is not evidence that all peers are current.

Do not delete acknowledgement rows to make a dashboard count appear healthy. To roll back an updater, disable its timer, restore the previous reviewed active bundle and matching service configuration, then restart and verify the exact target service. Preserve logs and the activation marker for diagnosis.

9. Current verification and release gates

Candidate checks on 23 October 2026: CRL activation, generator, and publisher tests passed 29/29 (18/5/6); API fixtures 3/3; Certificates UI, service-worker, JSX parsing (16/16 screens), and PHP syntax passed. CMake build succeeded and CTest passed 32/32 with `TMPDIR=/dev/shm`. Parser, redirect, first-activation, pre-replacement write-failure, and receipt-write recovery edges are covered. CI now EXPLAINs the dashboard's exact CRL queries against its migrated scratch database. The live DB target self-skipped locally; an earlier 17/17 disposable DB run predates the latest acknowledgement change. Cross-lab review, post-change live DB run, hosted CI, deployed HTTPS retrieval, peer activation, rollback drill, and rollout remain unverified.

The authoritative design and lifecycle contract is F-01 revocation lifecycle; implementation notes and commands are in deploy/crl/README.md. This manual is a candidate artifact and must be refreshed with verified deployment details before release.